**Cheatsheet made by Nikhil Rupala 🙂**

# Golden and Important Critical Exams to keep in mind while solving DSA

**Lab Manual - Lab Number 21** ના તમામ practicals (Programs 81 thi 83, sathe Connected Advanced Tree Concepts) mate no complete **High-Yield Concepts**, **Complexities**, ane **Golden Tips** niche given che:

## 1. Lab 21 Overview & Problem Breakdown

Lab 21 ma Advanced Binary Search Trees ane Construction Algorithms cover thay che:

- Problem 81: Construct Binary Tree from Preorder & Postorder Sequences
  - *Core Concept:* Preorder sequence mathi pehlo element root bane che, ane teno next element left subtree no root bane che. Postorder sequence ma left subtree na root ni position find kari left ane right subtree na boundaries split karay che.
- Problem 82: Smallest and Largest Elements in a BST
  - *Smallest Element:* Root thi start kari ne continuously **Leftmost leaf** (node->left == NULL) sudhi traverse karo.
  - *Largest Element:* Root thi start kari ne continuously **Rightmost leaf** (node->right == NULL) sudhi traverse karo.
- Problem 83: Phonebook Dictionary using BST
  - *Core Concept:* char name[100] ane char phone[15] dharavtu dynamic node structure.
  - *Operations:* Insert (Alphabetical comparison using strcmp), Delete (3 Cases: Leaf, Single Child, Inorder Successor for 2 Children), Search by Name, Ascending List (Inorder: Left → Root → Right), ane Descending List (Reverse Inorder: Right → Root → Left).
- Advanced Tree Implementations (Programs 78, 79, 80):
  - *Height-Balanced Tree (AVL Check):* |Height(Left) - Height(Right)| <= 1.
  - *Red-Black Tree:* 5 Invariant properties, Recoloring, Left/Right Rotations.
  - *2-3 Tree:* Multiway search tree, node split ane middle key promotion.

## 2. High-Yield Theoretical & Coding Concepts for Exams

### A. Finding Extrema in BST (Min/Max Key Extraction)

-

```C/C++
// Minimum element in BST
int findMin(struct Node* root) {
    if (root == NULL) return -1;
    while (root->left != NULL) root = root->left;
    return root->data;
}

// Maximum element in BST
int findMax(struct Node* root) {
    if (root == NULL) return -1;
    while (root->right != NULL) root = root->right;
    return root->data;
}
```

- *Time Complexity:* O(H), jya H height of tree che (O(log N) for balanced, O(N) for skewed tree).

### B. BST Deletion Mechanics (3 Golden Cases)

Exams ma deletion no code/dry-run sauthi vadhu marks ma puchay che:

- **Case 1 (0 Children / Leaf Node):** Directly free(root) kari pointer NULL return karo.
- **Case 2 (1 Child):** Target node na single child (root->left athva root->right) no address preserve karo, current node free() karo, ane child pointer return karo.
- **Case 3 (2 Children):** Right subtree mathi **Inorder Successor** (smallest value in right subtree) find karo, teno data current node ma copy karo, ane right subtree mathi Inorder Successor node recursively delete karo.

### C. Lexicographical / Dictionary Ordering using strcmp()

- strcmp(str1, str2) < 0 -> str1 dictionary ma pehla ave -> **Left Subtree** ma insert/search karo.
- strcmp(str1, str2) > 0 -> str1 dictionary ma pachi ave -> **Right Subtree** ma insert/search karo.
- strcmp(str1, str2) == 0 -> Key match thai gai (Duplicate found / Search hit).

### D. Ascending vs Descending Order Traversal

- **Ascending Order (A to Z):** Standard **Inorder Traversal** (Left -> Root -> Right).
- **Descending Order (Z to A): Reverse Inorder Traversal** (Right -> Root -> Left).

## 3. Comprehensive Time & Space Complexity Master Table

| Operation / Concept | Best Case | Average Case | Worst Case (Skewed) | Auxiliary Space |
|---|---|---|---|---|
| **Search Min/Max in BST** | O(1) | O(log N) | O(N) | O(1) (Iterative) |
| **Search by Name (Phonebook)** | O(1) | O(log N) | O(N) | O(H) (Recursion) |
| **Insert / Delete in Phonebook** | O(1) | O(log N) | O(N) | O(H) |
| **Inorder Traversal (Ascending/Descending)** | O(N) | O(N) | O(N) | O(H) |
| **Construct Tree (Preorder + Postorder)** | O(N) | O(N) | O(N^2) (Search) | O(N) |
| **Height Balanced (AVL) Validation** | O(1) | O(N) | O(N) | O(H) |

# 4. Critical Exam & MCQ Golden Points

1. **BST Inorder Traversal Property:** Binary Search Tree no Inorder traversal hamesha elements ne **Ascending (Sorted) Order** ma produce kare che.
2. **Reverse Inorder Output:** Binary Search Tree no Reverse Inorder traversal (Right -> Root -> Left) elements ne **Descending Order** ma produce kare che.
3. **Unique Binary Tree Construction Condition:** General Binary Tree ne uniquely construct karva mate **Inorder Traversal hovo mandatory (farjiyaat) che**. Preorder +

Postorder thi unique tree tyare j bane jo Binary Tree **Full/Strict Binary Tree** (darek node ne 0 athva 2 children) hoy.

4. **Inorder Successor Position:** Inorder Successor hamesha target node na **Right Subtree no Leftmost Node** hoy che (teno left child hamesha NULL j hoy).
5. **Memory Management during Deletion:** Dynamic BST mathi node delete karti vakhate memory leak avoid karva mate free() system call execute karvo essential che.